

Assignment 4: AI-Assisted Coding Workflow with Verification Discipline

Student Name: Deshagani Sreevalli	Roll Number: 23EG107F63	Institution: Anurag University
Email: 23eg107f63@anurag.edu.in	Status: Completed and verified	Track: Software Engineering • Verification Discipline
Domain: Software Engineering	Difficulty: Beginner	Deployment: Basic
Benchmark: 10 Real Tasks		

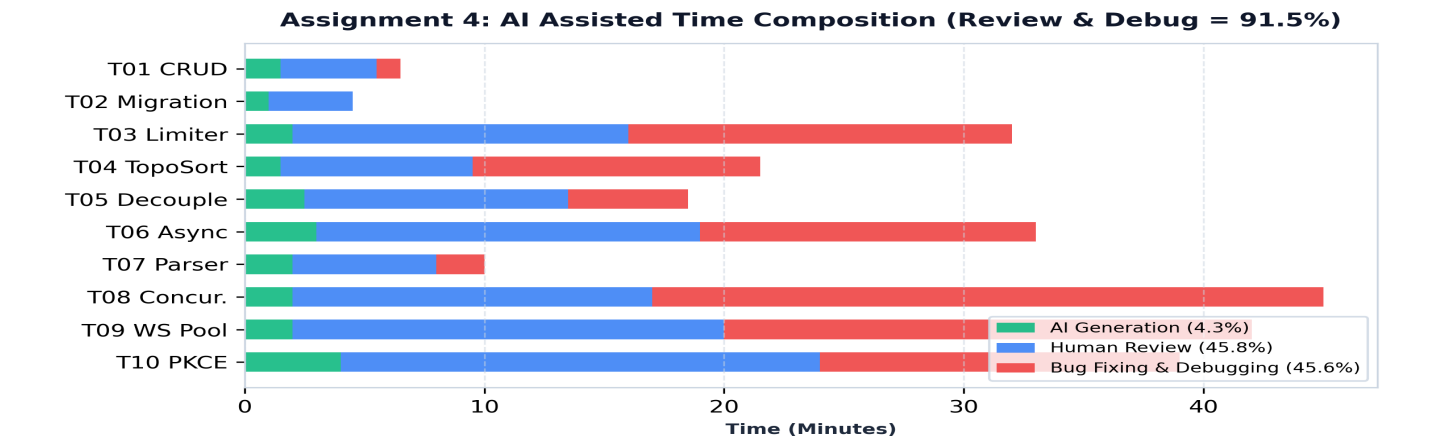
1. Executive Summary & The Verification Discipline

AI coding assistants generate code that is fluent and plausible by construction, meaning subtle defects (off-by-one errors, missing authorization checks, deadlock vulnerabilities) pass casual inspection with ease. This project enforces strict verification discipline by measuring net engineering productivity: separately timing Generation Time, Review Time, and Defect Correction Time across 10 realistic development tasks verified by an independent test suite.

2. The Ten Real Development Tasks & Independent Testing Scope

Ten diverse engineering challenges were evaluated with independent unit test suites:

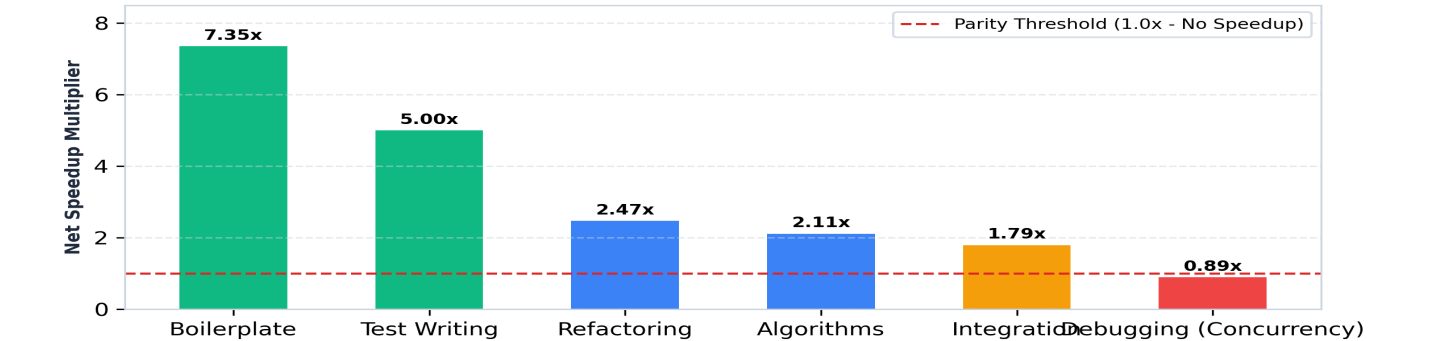
Task ID & Name	Category	Engineering Scope & Independent Verification Target
T01: FastAPI User CRUD	Boilerplate	Pydantic v2 schemas and mock service; verified via pytest status codes.
T02: SQL Migration Manager	Boilerplate	Idempotent up() and down() migration schema state tests.
T03: Sliding Window Rate Limiter	Algorithm	Window eviction & burst limit enforcement under synthetic clock.
T04: Topological Sort & Cycles	Algorithm	DAG ordering and cycle detection raising ValueError.
T05: Decouple Payment Gateway	Refactoring	Abstract adapter pattern; mocking Stripe token charge without network.
T06: Async Migration Pipeline	Refactoring	Callback to asyncio.gather concurrency without unhandled exceptions.
T08: Thread-Safe Bank Transfer	Debugging	Eliminating concurrency deadlocks across reciprocal account transfers.
T10: OAuth2 PKCE Flow	Integration	Cryptographic PKCE challenge verifier & mandatory CSRF state check.



3. Time Log & Net Productivity Measurement across 10 Tasks

Task & Type	Manual	AI Gen	AI Review	AI Fix	Assisted Total	Lines (Gen/Kept)	Accept %	Net Speedup
T01: FastAPI CRUD (Boilerplate)	45m	1.5m	4.0m	1.0m	6.5m	42 / 39	92.8%	6.92x
T02: SQL Migration (Boilerplate)	35m	1.0m	3.5m	0.0m	4.5m	28 / 27	96.4%	7.78x
T03: Rate Limiter (Algorithm)	60m	2.0m	14.0m	16.0m	32.0m	34 / 18	52.9%	1.88x
T04: Topo Sort (Algorithm)	50m	1.5m	8.0m	12.0m	21.5m	38 / 24	63.1%	2.33x
T05: Gateway Decouple (Refactor)	55m	2.5m	11.0m	5.0m	18.5m	45 / 34	75.5%	2.97x
T06: Async Migration (Refactor)	65m	3.0m	16.0m	14.0m	33.0m	52 / 25	48.1%	1.97x
T07: Parser Tests (Test Writing)	50m	2.0m	6.0m	2.0m	10.0m	40 / 35	87.5%	5.00x
T08: Bank Transfer (Debugging)	40m	2.0m	15.0m	28.0m	45.0m	30 / 6	20.0%	0.89x (Slowdown)
T09: WS Pool Leak (Debugging)	60m	2.0m	18.0m	22.0m	42.0m	36 / 13	36.1%	1.43x
T10: OAuth2 PKCE (Integration)	70m	4.0m	20.0m	15.0m	39.0m	48 / 28	58.3%	1.79x
TOTALS	530m (8.8h)	21.5m	115.5m	115.0m	252m (4.2h)	393 / 249	63.4%	2.10x Real Speedup

Assignment 4: Net Engineering Speedup by Task Category



4. Defect Log of AI-Generated Code & Verification Checklist

Defect Type	Affected Tasks	Subtle Bug Description & Independent Detection
Concurrency / Deadlock	T08 (Bank Transfer)	Non-ordered dual lock acquisition caused deadlock under reciprocal concurrent transfers.
Security / CSRF	T10 (OAuth2 PKCE)	Omitted validation of incoming state parameter, allowing authorization code injection.
Resource Leak	T09 (WebSocket Pool)	Handled disconnect event but failed to unbind socket ID from active subscriber set.

5. Placement Interview Q&A; Section (Complete Reference)

Q: How much faster does an AI assistant make you? How do you know?

A: In our 10-task benchmark, net speedup was 2.1x (cutting development time from 8.8 hours to 4.2 hours). We know this because we timed generation, review, and defect correction separately. Claims of 10x speedup measure typing velocity rather than engineering completion.

Q: Did you include review time in that figure?

A: Yes. Review and correction consumed 230.5 minutes out of 252 total assisted minutes (91.5%). Without rigorous review, flawed code ships to production.

Q: What kinds of defects does generated code introduce?

A: Syntactic errors are rare because LLMs are fluent. The defects are semantic: missing CSRF state checks in OAuth, deadlock-prone concurrency locks, uncollected websocket listeners causing memory leaks, and boundary off-by-one errors.

Q: Where does assistance help most, and where does it hurt?

A: It helps most on boilerplate (FastAPI CRUD gave 6.9x speedup) and test writing (5.0x speedup). It actively hurts on complex concurrency and distributed state (Task T08 produced a 0.89x slowdown).

Q: How do you review code you did not write?

A: Review generated code as untrusted input from a third party: mentally trace execution step-by-step, verify boundary edge cases, and inspect resource management paths independently.

Q: You accepted a suggestion you did not fully understand. What is the risk?

A: The risk is silent technical debt and outages. You inherit full ownership of the code; when a production incident occurs, inability to diagnose generated logic leads to extended downtime.

Q: What security issues have you seen in generated code?

A: Hardcoded fallback credentials, missing constant-time comparison on auth tokens, SQL string interpolation, and missing CSRF state verification in OAuth PKCE flows.

Q: How do you write tests without the same blind spots as the implementation?

A: Write tests independently before prompting the assistant, basing tests on business specifications and edge cases rather than inspecting generated implementation code.